

Building An Agentic Pipeline with Codex

A reproducible multi-agent reviewer for economics papers

Ingar Haaland

Norwegian School of Economics

What You Will Be Able To Do

Session goals

- ▶ Read the basic shape of a research-code repository.
- ▶ Distinguish agents, prompt templates, skills, schemas, scripts, and rosters.
- ▶ Explain why agentic work needs artifacts, validation, and traceability.
- ▶ Transfer the pattern to your own economics workflows.

Running case

A local pipeline that takes an economics paper PDF and produces a structured, traceable referee-style review.

What Is A Repository?

Plain-language definition

A repository is a project folder with memory: it keeps source files, instructions, generated artifacts, and change history together.

- ▶ Code and prompts live next to documentation.
- ▶ Configuration makes repeated behavior explicit.
- ▶ Intermediate artifacts are inspectable rather than hidden in chat history.
- ▶ Version control records what changed and why.

Term for later: an agent roster is the run catalog that connects workers, prompts, outputs, stages, and mandatory or optional review policy.

Source Files Versus Generated Artifacts

Source files

Files we edit and keep as project machinery.

- ▶ scripts
- ▶ prompt templates
- ▶ schemas
- ▶ reviewer roster
- ▶ repo guidance

Generated artifacts

Files produced by one run.

- ▶ parsed pages
- ▶ rendered prompts
- ▶ selected reviewer roster
- ▶ reviewer JSON
- ▶ final report

Basic Folder Map

Location	Role
inputs/	source PDFs
work/.../parsed/	extracted evidence surface
work/.../prompts/	run-specific agent assignments
work/.../reviews/	structured reviewer JSON outputs
work/.../selection/	saved reviewer-selection decision
work/.../editor/	normalized evidence and editor input
outputs/.../report	final report

The folder map is also the audit trail

Each stage leaves a file you can inspect, rerun, or cite when something goes wrong.

The Folder Layout Is The Pipeline

Core idea

The pipeline is not a conversation. It is a sequence of artifacts: PDF → parsed evidence → prompts → reviewer JSON → editor input → report.

- ▶ Code creates stable intermediate files.
- ▶ Agents inspect bounded artifacts rather than an unbounded PDF.
- ▶ Validation catches malformed outputs before synthesis.
- ▶ The final report can trace claims back to source reviewer findings.

Economics Tasks Need Code And Judgment

Code is good at

- ▶ extracting pages and tables
- ▶ checking schemas
- ▶ normalizing outputs
- ▶ routing files

Agents are useful for

- ▶ claim-evidence matching
- ▶ identifying confounds
- ▶ literature-positioning checks
- ▶ prioritizing reviewer findings

The paper reviewer combines both: deterministic scripts prepare evidence; specialist agents audit judgment-heavy questions.

Why One-Off Chat Fails As A Referee Workflow

- ▶ No stable input state: the model may not see the same evidence twice.
- ▶ No stable roles: every check competes inside one broad prompt.
- ▶ No output contract: prose cannot be reliably validated or merged.
- ▶ No traceability: claims in the report are hard to map back to source pages.
- ▶ No rerun boundary: a failed check often means starting over.

Insight

Agentic research work becomes reliable only when the conversation is turned into a file-based workflow.

The Pipeline Version: One Complete Run

#	Stage	File checkpoint and purpose
1	PDF intake	Keep the raw source in <code>inputs/paper3.pdf</code> ; never edit it.
2	Parse PDF	Create page text, table, figure, and inventory files in <code>work/paper3/parsed/</code> .
3	Parser preflight	Run <code>parser_quality_auditor</code> ; stop or caveat if extraction is unsafe.
4	Render prompts	Fill run-specific assignments under <code>work/paper3/prompts/</code> .

#	Stage	File checkpoint and purpose
5	Select roster	Save the chosen mandatory and optional reviewers in <code>selection/</code> .
6	Review	Each agent writes one validated JSON file in <code>work/paper3/reviews/</code> .
7	Normalize	Merge duplicates and build the editor bundle in <code>work/paper3/editor/</code> .
8	Synthesize	The editor writes <code>outputs/paper3/report.md</code> ; smoke checks catch broken output.

What changed relative to one-off chat

Every stage has a named input, output, and rerun boundary, so quality checks happen before synthesis rather than after-the-fact.

What Is An Agentic Model?

Definition for this session

An agentic model is a language model used as a bounded worker in a larger loop: it receives a role, tools or artifacts, a task, and an output contract, then its output changes the next step of the workflow.

What makes this pipeline agentic

- ▶ The selector chooses which optional reviewers should run.
- ▶ Specialist reviewers inspect different evidence classes.
- ▶ The editor synthesizes validated reviewer findings.
- ▶ Scripts use those agent outputs to decide what to validate, normalize, and report.

From Referee Task To Pipeline

Referee task	Pipeline version
Read the paper	parse PDF into page-level artifacts
Check whether evidence is usable	parser-quality preflight
Decide which checks matter	dynamic reviewer selection
Inspect numbers, claims, references	specialist reviewer agents
Combine findings	normalize, dedupe, and route issues
Write the report	editor agent plus traceability appendix

Design pattern

Turn a vague expert task into a chain of small, inspectable contracts.

Codex Files: Who Does What

File	Metaphor	Example in this repo
AGENTS.md	constitution	rules for preprocessing, review, and report writing
.codex/config.toml	defaults	model, sandbox, approval, project root
.codex/agents/*.toml	workers	parser, numerical, claim-evidence, editor agents
skills/.../SKILL.md	playbook	the paper-reviewer workflow recipe
config/reviewers.json	roster	enabled reviewers and run options
prompt templates	assignment pattern	exact task given to each worker
schemas/*.json	contract	valid JSON shape
scripts/*.py	glue	deterministic orchestration

Roster: the run-time catalog that says which agents are available, when they run, and what files they produce.

Translation Dictionary

Software language	Organization language
▶ custom agents	▶ workers
▶ reviewer config	▶ roster and run options
▶ skills	▶ playbooks
▶ prompt templates	▶ assignments
▶ schemas	▶ output contracts
▶ scripts	▶ operations team

The paper reviewer is a small organization, not one giant prompt.

Agents, Prompt Templates, Skills: Do Not Mix These Up

Agent = worker

`numerical_auditor`

Role identity and behavior:
checks numeric claims, signs,
units, and rounding.

Skill = playbook

`paper-reviewer`

Procedure for Codex: use the
wrapper, validate JSON,
normalize, and run the editor.

Prompt template = assignment

`numerical_audit.txt`

Run-specific task text: audit
paper3 artifacts and return
NUM-001 style JSON.

Roster connection

`reviewers.json` binds these pieces together: agent name, prompt file, output file, stage, search flag, and mandatory/optional policy.

What it does

AGENTS.md is the repo constitution. It tells Codex how the project should behave before any one task begins.

- ▶ Preprocess before review.
- ▶ Never run reviewers directly on raw PDFs.
- ▶ Use structured JSON for internal reviewers.
- ▶ Only the editor writes the final markdown report.
- ▶ Keep source locations and traceability.

Actual File: AGENTS.md

The normal workflow is:

1. Put source PDFs in 'inputs/'.
2. Preprocess each paper into structured artifacts under 'work/<paper_id>/parsed/'.
3. Render run-specific prompts under 'work/<paper_id>/prompts/'.
4. Run parser-quality preflight before substantive review.
5. Use dynamic reviewer selection by default to choose optional reviewers while mandatory reviewers always run.
- ...

Reviewer agents are configured through 'config/reviewers.json'.

Internal reviewer agents return structured JSON only.

Only the editor writes the final markdown report.

Why this matters

The rules prevent the pipeline from silently skipping preprocessing, inventing evidence, or mixing draft review prose with final report prose.

Project Defaults: `.codex/config.toml`

What it does

Project defaults make repeated Codex sessions start from the same assumptions: model, reasoning effort, approval policy, sandbox, and project root markers.

Why it matters

Reproducibility is not just about scripts. It also depends on stable execution defaults and explicit permission boundaries.

Actual File: .codex/config.toml

```
model = "gpt-5.4"
model_reasoning_effort = "high"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"
personality = "none"
project_root_markers = [".git"]

[windows]
sandbox = "unelevated"
```

Interpretation

The project asks for high-reasoning review work, workspace-limited file access, and explicit approval for operations outside the sandbox.

Custom Agents: Workers With Narrow Jobs

What an agent file does

A custom agent defines a role, a scope, and behavior constraints. It is a worker identity, not the full run-specific assignment.

- ▶ `parser_quality_auditor`: checks whether parsed evidence is safe.
- ▶ `numerical_auditor`: checks numerical claims against tables and figures.
- ▶ `claim_evidence_auditor`: checks whether textual claims match displayed evidence.
- ▶ `editor`: writes the final synthesis from validated reviewer evidence.

Actual File: parser_quality_auditor.toml

```
name = "parser_quality_auditor"
description = "Checks parsed PDF artifacts for extraction, ordering, caption, table,
and inventory defects before downstream review."
model_reasoning_effort = "medium"

developer_instructions = """
You are the parser quality auditor.

Your job is to audit preprocessing artifacts under work/<paper_id>/parsed/
before substantive reviewers rely on them.

Check:
- missing page artifacts
- likely scanned or low-text pages
- scrambled reading order
- broken tables
- missing or detached captions
...
- return JSON only
"""
```

Reviewer Roster: `config/reviewers.json`

What a roster adds

The roster defines how each reviewer participates in a run: role, prompt, output file, stage, search setting, and selection policy.

What each roster entry binds together

agent name + prompt template + output JSON + id prefix + search flag + stage + mandatory/optional policy

Mandatory

Always run when enabled. Examples: parser preflight, cross-reference, reference integrity, grammar.

Optional

Available to the selector when the paper has matching risks. Examples: numerical, identification, robustness, sample construction.

Configured Reviewer Agents

P = appendix slide with the actual prompt template. Exact agent IDs live in `config/reviewers.json`.

Mandatory: always run			Optional selector pool			Optional selector pool		
Agent	Job	P	Agent	Job	P	Agent	Job	P
Parser quality	extraction safety	P	Numerical	signs, units, rounding	P	Limits/external	scope, welfare	P
Cross-reference	tables, figures, sections	P	Claim evidence	claims vs. evidence	P	Model/equation	notation, assumptions	P
References	bibliography integrity	P	Literature	novelty positioning	P	Replication	preregistration	P
Grammar	copyediting triage	P	Identification	causal assumptions	P	Institutional	legal, market context	P
			Robustness	specs and subsamples	P	Power/testing	power, multiplicity	P
			Sample	exclusions, attrition	P	Design/random.	assignment, spillovers	P
			Abstract/conclusion	front/back consistency	P	Economic magnitude	baselines, welfare	P

Actual File: config/reviewers.json

```
{
  "name": "numerical_auditor",
  "prompt": "numerical_audit.txt",
  "output": "numerical_auditor.json",
  "id_prefix": "NUM",
  "search": false,
  "enabled": true,
  "normalization_role": "manuscript",
  "selection_policy": "optional"
},
{
  "name": "reference_auditor",
  "prompt": "reference_audit.txt",
  "output": "reference_auditor.json",
  "id_prefix": "REF",
  "search": true,
  "enabled": true,
  "normalization_role": "reference",
  "selection_policy": "mandatory"
}
```

Roster logic

The same JSON file both lists workers and tells the wrapper how to route their outputs.

Skills: Playbooks For Repeated Work

What a skill does

A skill is a reusable procedure for Codex: when to use it, what steps to follow, and which scripts or artifacts matter.

Paper-reviewer skill

The repo skill says: use `scripts/review_paper.py` for fresh runs, follow the full pipeline, use editor-only refresh only when reviewer evidence already exists, and preserve traceability.

Skills do not replace agents. They tell Codex how to orchestrate the work.

Actual Skill: paper-reviewer/SKILL.md

Workflow

For fresh runs, use 'scripts/review_paper.py' as the primary entry point.

The pipeline stages are:

1. Resolve the input PDF path.
2. Derive 'paper_id' from the filename stem unless explicitly provided.
3. Preprocess the PDF into 'work/<paper_id>/parsed/'.
4. Render run-specific prompts into 'work/<paper_id>/prompts/'.
5. Launch preflight reviewers from 'config/reviewers.json'.
- ...
14. Run the editor to write 'outputs/<paper_id>/report.md'.
15. Smoke-check the final report with 'scripts/check_final_report.py'.

Difference from an agent

The skill is the playbook. The agents are the people who do bounded parts of the playbook.

Further Skills This Project Could Use

Skill idea	Role
artifact-audit	inspect parsed artifacts before launching expensive reviewer runs
report-export	convert final Markdown reports to PDF, HTML, PNG, or teaching handouts
replication-package	package prompts, schemas, logs, and outputs for sharing
ci-triage	diagnose failing tests or workflow checks after pipeline changes

Do not confuse with agents

A skill is a reusable procedure for Codex. An agent is a specialist worker. A prompt template is the assignment given to that worker for one run.

Prompt Templates: Assignment Patterns

What a prompt template does

A prompt template is the reusable assignment text. It has placeholders for the paper ID, parsed directory, schema path, and reviewer-specific instructions.

- ▶ It tells the agent what to audit.
- ▶ It tells the agent where evidence lives.
- ▶ It tells the agent which JSON schema to satisfy.
- ▶ It tells the agent how to name finding IDs.

Rendered Prompt Example: paper3

Actual assignment excerpt

Use the custom agent named `numerical_auditor`.

Audit: `work\paper3\parsed`

Return one JSON object that strictly conforms to `schemas\reviewer_output.schema.json`.

Run-specific constraints

- ▶ `reviewer = numerical_auditor`
- ▶ `paper_id = paper3`
- ▶ finding IDs must be NUM-001, NUM-002, ...

Audit focus

Numeric claims in text versus tables, figures, appendix; signs, scales, units, rounding; and `cannot_verify` when artifacts are insufficient.

Schemas: Output Contracts

Why schemas matter

Free-form prose is hard to validate. Structured JSON lets the pipeline check whether reviewer evidence can be normalized, deduplicated, and traced.

- ▶ Finding IDs are stable: NUM-001, CEA-002, PARSER-001.
- ▶ Issue type is explicit: manuscript issue, parser artifact, cannot verify, copyedit issue.
- ▶ Each finding carries severity, confidence, location, evidence, and suggested fix.
- ▶ Some reviewers add specialized fields such as `numeric_check`.

Schema Example: Reviewer Output

What the schema forces every reviewer to provide

reviewer, paper_id, run_status, summary, findings, and notes.

Each finding must include

id, issue_type, severity, confidence, location, evidence_summary, source_objects, claim_evidence_links, numeric_check, and suggested_fix.

```
"issue_type": ["manuscript_issue", "parser_artifact",  
               "reference_integrity", "copyedit_issue", "cannot_verify"]  
"severity": ["high", "medium", "low"]  
"location.precision": ["exact", "partial", "missing"]
```

Schema Example: Reviewer Selection

The selector also has a contract

It cannot just write prose. It must classify the paper and justify which optional reviewers are selected or skipped.

```
"paper_type": [  
  "empirical_causal", "empirical_descriptive", "theory",  
  "methods", "literature_review", "mixed", "unknown"  
]  
"selection_confidence": ["high", "medium", "low"]  
"selected_optional_reviewers": [  
  {"name": "...", "reason": "..."}  
]
```

OpenAI Best Practices In This Pipeline

What transfers directly

Official OpenAI guidance emphasizes reusable prompts, explicit instructions, structured evaluation, and human review gates. This project implements those ideas as local files and checks.

- ▶ Version and reuse prompts: templates live in `prompts/templates/` and render into run-specific assignments.
- ▶ Separate stable instructions from variables: paper ID, parsed path, schema path, and reviewer role are injected per run.
- ▶ State success criteria explicitly: each reviewer receives scope, evidence boundaries, ID format, and JSON schema requirements.
- ▶ Evaluate behavior over time: completed runs become test cases for prompt, schema, and roster revisions.
- ▶ Keep human review gates around risky judgments: parser preflight, cannot-verify labels, and final report smoke checks make uncertainty visible.

Scripts: Deterministic Glue

What scripts do

Scripts perform repeatable operations that should not depend on model judgment.

Script	Job
<code>preprocess_pdf.py</code>	create parsed evidence artifacts
<code>render_prompts.py</code>	turn templates into run-specific prompts
<code>validate_review_json.py</code>	validate reviewer outputs
<code>normalize_findings.py</code>	merge and deduplicate findings
<code>build_editor_input.py</code>	assemble the editor brief
<code>check_final_report.py</code>	smoke-check final report structure and traceability

Fresh-Run Workflow: Exact Order

Step	Action	Appendix
1	Preprocess PDF into parsed artifacts	Command
2	Render prompts from templates	Prompt
3	Run parser-quality preflight	Preflight
4	Dynamically select optional reviewers	Selector
5	Run mandatory and selected reviewers	Roster
6	Validate, normalize, dedupe, and build editor input	Bundle
7	Run editor and smoke-check report	Report

Why buttons matter in teaching

The high-level pipeline stays readable, but each stage can jump to the exact command or instruction behind it.

Fresh Run: One Command, Many Artifacts

```
python scripts\review_paper.py inputs\paper3.pdf --paper-id paper3
```

The command is a wrapper

It does not make the project opaque. It writes intermediate artifacts under work/paper3/ so each stage can be inspected.

- ▶ work/paper3/parsed/
- ▶ work/paper3/prompts/
- ▶ work/paper3/selection/
- ▶ work/paper3/reviews/
- ▶ work/paper3/editor/
- ▶ outputs/paper3/report.md

What Evidence Do Reviewers Actually See?

Parsed page evidence

- ▶ normalized page text
- ▶ page labels
- ▶ sections
- ▶ raw blocks
- ▶ page images

Structured object evidence

- ▶ table inventory
- ▶ figure inventory
- ▶ extracted tables
- ▶ figure text or images
- ▶ manifest summary

Reviewers audit the artifact surface, not the raw PDF directly.

First Check The Evidence Surface

Insight

If the parser misses appendix tables or scrambles a figure caption, downstream reviewers may make confident claims from incomplete evidence.

- ▶ Parser preflight runs before substantive review.
- ▶ Parser caveats are preserved into the final report.
- ▶ High-confidence blocking parser failures should stop the workflow.
- ▶ Lower-severity parser defects become auditability caveats.

Paper3: Reviewer Selection Output

Saved decision

`work/paper3/selection/reviewer_selection.json`

- ▶ `paper_type`: empirical causal
- ▶ `selection_confidence`: high
- ▶ selected numerical, claim-evidence, literature, identification, robustness, sample-construction, external-validity, replication, power, design, and magnitude reviewers
- ▶ skipped model-equation and institutional-context reviewers

Why this is better than ad hoc prompting

The decision is saved, inspectable, and rerunnable; it is not buried in chat state.

Paper3: Selection Rationale

Selector classification

paper_type: empirical causal

selection_confidence: high

Why numerical auditor was selected

The parsed paper contains dense quantitative evidence: tables, figures, percentages, treatment-effect estimates, appendix estimates, and reported p-values.

Why identification auditor was selected

The paper is built around randomized field experiments, cross-randomized treatment arms, and treatment-effect claims.

Selector output

The selector is itself an agentic step: it classifies the paper and changes which workers run next.

One Worker, One Evidence File

Modularity

Each reviewer writes one structured JSON file under `work/paper3/reviews/`.

- ▶ A reviewer can fail independently.
- ▶ A reviewer can be inspected independently.
- ▶ A reviewer can be rerun independently.
- ▶ The editor can cite exactly which reviewer findings support a synthesis point.

Paper3

`numerical_auditor.json`, `claim_evidence_auditor.json`,
`parser_quality_auditor.json`, and thirteen other reviewer outputs were written.

A Finding Is An Auditable Claim

Field	Why it matters
id	stable handle for deduplication and traceability
issue_type	separates manuscript issues, parser artifacts, and cannot-verify cases
location	tells the editor where the issue appears
evidence_summary	compact rationale
source_objects	links claim to parsed pages, tables, figures, or external sources
suggested_fix	makes the report actionable

Output contract

The reviewer is not just "giving feedback"; it is producing evidence that another stage can validate and use.

Paper3 Reviewer Output: Numerical Auditor

Finding NUM-001

Page 32, Section 4.4.1: the manuscript says respondents who believe news is too conservative spend around **0.30 SD** more time using the Liberal setting.

Auditor assessment

Figure 8 places the relevant coefficients only a little above **0.20 SD**; the plotted magnitude is materially smaller than the text.

Suggested fix

Revise the text to about 0.2 standard deviations, or report the exact regression coefficient.

Paper3 Reviewer Output: Claim-Evidence Auditor

Finding CEA-002

The paper claims that a majority favor politically neutral reporting.

Auditor assessment

The displayed evidence is an aggregate time-share plot, not a respondent-majority statistic. The highest neutral/fact-focused share shown is **46%**, not a majority.

Suggested fix

Recast the result in terms of displayed time shares, or add respondent-level choice shares.

Example of judgment

This is not a string match. The agent compares claim, denominator, displayed evidence, and interpretation.

Normalization: From Many Findings To Canonical Issues

Why normalization is needed

Several reviewers may identify overlapping versions of the same problem.

- ▶ Normalize fields across reviewer outputs.
- ▶ Deduplicate overlapping findings.
- ▶ Preserve source reviewer IDs.
- ▶ Carry severity, confidence, parser caveats, and cannot-verify labels forward.

Paper3 bundle

`normalized_bundle.json`: 60 canonical findings, including 31 manuscript issues, 8 parser artifacts, 5 cannot-verify items, and 9 copyedit issues.

Scripts

Improving Reviewer Quality

Human validation is part of the system

The pipeline can make reviewer behavior inspectable, but humans still need to evaluate whether each specialist is useful, calibrated, and worth running.

- ▶ Score findings against human labels: correct, overstated, duplicate, low-value, or wrong.
- ▶ Track which reviewers produce high-signal findings by paper type.
- ▶ Refine prompts when reviewers miss evidence or overclaim.
- ▶ Update schemas and validation when output failures repeat.
- ▶ Use completed runs such as `paper3` as regression cases before changing prompts or models.

Quality loop

Run → inspect reviewer JSON → label against human audit → refine prompt/roster/schema → rerun.

Editor Input: Deterministic Brief Plus Evidence

What the editor receives

The editor does not start from scratch. It receives a deterministic brief, the normalized bundle, and the original reviewer JSON files.

- ▶ Which findings deserve priority?
- ▶ Which issues are duplicates?
- ▶ Which caveats come from parser artifacts?
- ▶ Which findings belong in appendices?
- ▶ Which source IDs must appear in the traceability map?

Paper3 Editor Input: Hard Rules

Hard constraints

- ▶ Do not invent findings.
- ▶ Preserve uncertainty, especially `cannot_verify` cases.
- ▶ Preserve exact locations when available.
- ▶ Merge duplicate findings into one canonical issue.
- ▶ Start with cross-agent synthesis, not agent-by-agent sections.

Traceability requirement

Include a final appendix mapping every canonical finding in the normalized bundle to its source reviewer finding IDs.

Editor role

The editor writes synthesis, but only from validated evidence.

Final Report: Referee-Style Synthesis

Paper3 report structure

- ▶ Executive Summary
- ▶ Review Configuration
- ▶ Highest-Priority Cross-Agent Findings
- ▶ Suggested Revision Priorities
- ▶ Additional Findings
- ▶ Parser and Preprocessing Caveats
- ▶ Items Marked Cannot Verify
- ▶ Traceability Map

Report principle

The final report is prose for humans, but its claims still map back to structured reviewer evidence.

Paper3 Final Report: Example Synthesis

Editor synthesis excerpt

The strongest concerns are that the abstract and conclusion generalize from a selected AI-news-app sample to "most people"; that the Facts Only arm is treated as if it cleanly identifies neutral-content preferences; and that some headline result statements go beyond the evidence shown.

What makes this synthesis

The paragraph merges abstract/conclusion, external-validity, identification, claim-evidence, and magnitude findings into one prioritized referee narrative.

What changed from raw reviewer JSON

The editor turns many modular findings into a prioritized referee narrative.

Reliability Layers Reduce Risk

Repo guidance and skill playbook

Deterministic preprocessing and prompt rendering

Narrow custom agents

JSON schemas and semantic validation

Normalization and editor brief

Final report smoke check and traceability

Important limit

These layers reduce avoidable errors. They do not prove that the final review is correct.

cannot_verify Is A Feature

Meaning

A reliable reviewer should say when available artifacts do not support a check.

- ▶ It prevents fake precision.
- ▶ It tells humans what evidence is missing.
- ▶ It separates manuscript uncertainty from parser limitations.
- ▶ It gives the editor a safer way to preserve uncertainty.

Paper3 example

Some exact figure-based numeric ranges could not be verified because the parsed artifacts exposed plots but not machine-readable coefficients.

Smoke Checks Are Not Substantive Audits

What smoke checks can test

- ▶ report exists
- ▶ required sections exist
- ▶ traceability appendix includes expected finding IDs
- ▶ output is not obviously a process note

What they cannot test

A smoke check cannot prove that the literature is current, the econometrics are right, or the editor prioritized the correct issues.

Human Audit Boundary

- ▶ Humans decide whether the automated review is useful.
- ▶ Humans decide whether the reviewer roster is adequate.
- ▶ Humans decide which findings matter for a real referee report.
- ▶ Humans decide when parser artifacts are too weak to proceed.

Insight

Agentic pipelines should make judgment more inspectable, not pretend to remove it.

Known Weaknesses

- ▶ PDF parsing remains fragile, especially tables and appendix objects.
- ▶ Dynamic selection can choose only among configured reviewers.
- ▶ JSON validation checks structure, not truth.
- ▶ External verification depends on search availability and source quality.
- ▶ The final report can still over- or under-prioritize issues.

Practical response

Keep artifacts inspectable, keep reviewers modular, and preserve uncertainty.

Where This Transfers In Economics

Workflow	Agentic pipeline version
Literature review	claim extraction, source verification, synthesis with citation traceability
Replication package	artifact inventory, script runner, failure classifier, report generator
Data appendix audit	table checker, denominator checker, cross-reference checker
Grant review	eligibility preflight, scoring agents, panel synthesis
Course material	examples, quizzes, slide compilation, consistency checks

Reusable recipe

Use scripts for repeatable operations and agents for bounded judgment.

Build One Pipeline Card

Template

- ▶ Input artifact:
- ▶ Deterministic preprocessing:
- ▶ Specialist judgment checks:
- ▶ Output contract:
- ▶ Validation gate:
- ▶ Final synthesis:
- ▶ Traceability mechanism:

Question

Where would you most want a saved artifact rather than a chat answer?

Reading A Repo Like This One

1. Read `README.md` for the mental model.
2. Read `AGENTS.md` for project rules.
3. Inspect `config/reviewers.json` for the roster.
4. Inspect `prompts/templates/` for assignments.
5. Inspect `schemas/` for output contracts.
6. Follow one completed run under `work/paper3/`.
7. Compare final report claims with reviewer JSON and source objects.

Closing Message

Main takeaway

An agentic research pipeline is a file-based organization of model workers, not a better chat prompt.

Design rule

Put deterministic code around model judgment: artifacts before judgment, schemas before prose, validation before synthesis, and traceability before trust.

Appendix: Wrapper Command

```
python scripts\review_paper.py inputs\paper3.pdf --paper-id paper3
```

- ▶ Resolves PDF path.
- ▶ Preprocesses into work/paper3/parsed/.
- ▶ Renders prompts.
- ▶ Runs selector, reviewers, validation, normalization, editor, and smoke check.

[Back to workflow](#)

Appendix: Rendered Prompt

Use the custom agent named numerical_auditor.

Audit:

`'work\paper3\parsed'`

Return one JSON object that strictly conforms to:

`'schemas\reviewer_output.schema.json'`

For this run:

- reviewer = "numerical_auditor"
- paper_id = "paper3"
- finding IDs must be stable and formatted NUM-001, NUM-002, ...

[Back to workflow](#)

Appendix: Parser Preflight Instructions

Your job is to audit preprocessing artifacts under `work/<paper_id>/parsed/` before substantive reviewers rely on them.

Check:

- missing page artifacts
- likely scanned or low-text pages
- scrambled reading order
- broken tables
- missing or detached captions
- figure/table extraction failures
- parser defects that would poison downstream reviewers

[Back to workflow](#)

Appendix: Dynamic Reviewer Selection

Use the custom agent named `reviewer_selector`.

Classify `paper_id "{paper_id}"` using parsed artifacts under:
`{parsed_dir}`

Select optional reviewers only from this catalog:

```
```json
{optional_reviewer_catalog}
```
```

Selection rules:

- Select `numerical_auditor` for papers with important numeric claims, tables, figures...
- Select `identification_auditor` for empirical_causal papers...
- Select `robustness_auditor` for empirical papers with robustness checks...
- Select `sample_construction_auditor` for nontrivial sample restrictions...

[Back to workflow](#)

Appendix: Roster Entry

```
{  
  "name": "numerical_auditor",  
  "prompt": "numerical_audit.txt",  
  "output": "numerical_auditor.json",  
  "id_prefix": "NUM",  
  "search": false,  
  "enabled": true,  
  "normalization_role": "manuscript",  
  "selection_policy": "optional"  
}
```

[Back to workflow](#)

Appendix: Parser Quality Prompt

Actual prompt template

parser_quality_audit.txt: use parser_quality_auditor; audit parser quality only; return reviewer JSON.

Focus excerpt

Missing page artifacts, low-text pages, scrambled reading order, broken tables, missing captions, figure/table extraction failures, and parser defects that could poison downstream reviewers.

[Back to agent overview](#)

Appendix: Crossref Prompt

Actual prompt template

crossref_audit.txt: use crossref_auditor; audit internal cross-references; return reviewer JSON.

Focus excerpt

Check whether references to tables, figures, equations, appendices, and sections resolve to the objects the text claims.

[Back to agent overview](#)

Appendix: Reference Prompt

Actual prompt template

reference_audit.txt: use reference_auditor; audit reference integrity; web search is enabled by the roster.

Focus excerpt

Bibliography accuracy, missing references, duplicate entries, citation metadata, and working-paper currency.

[Back to agent overview](#)

Appendix: Grammar Prompt

Actual prompt template

`grammar_audit.txt`: use `grammar_auditor`; audit grammar and copyediting issues only.

Focus excerpt

High-signal reader-facing mistakes: grammar, typo, punctuation, word choice, and sentence-level clarity issues for the appendix.

[Back to agent overview](#)

Appendix: Numerical Prompt

Actual prompt template

`numerical_audit.txt: use numerical_auditor; audit work/paper3/parsed; return NUM-001 style JSON.`

Focus excerpt

Numeric claims in text versus tables, figures, appendix; sign, scale, units, rounding; percentage versus percentage-point distinctions; `cannot_verify` when artifacts are insufficient.

[Back to agent overview](#)

Appendix: Claim-Evidence Prompt

Actual prompt template

`claim_evidence_audit.txt`: use `claim_evidence_auditor`; compare major text claims with displayed evidence.

Focus excerpt

Whether substantive empirical, policy, or interpretive claims are supported, contradicted, partially supported, or cannot be verified.

[Back to agent overview](#)

Appendix: Literature Prompt

Actual prompt template

literature_audit.txt: use literature_auditor; audit novelty claims and literature positioning.

Focus excerpt

Characterization of prior work, firstness or novelty claims, and whether cited studies support the paper's positioning.

[Back to agent overview](#)

Appendix: Identification Prompt

Actual prompt template

identification_audit.txt: use identification_auditor; audit identification and causal interpretation only.

Focus excerpt

Whether causal claims match research design, assumptions, threats, interpretation, and estimands.

[Back to agent overview](#)

Appendix: Robustness Prompt

Actual prompt template

robustness_audit.txt: use robustness_auditor; audit robustness and sensitivity support only.

Focus excerpt

Whether main empirical claims hold across robustness checks, specifications, subsamples, placebo tests, and appendix validations.

[Back to agent overview](#)

Appendix: Sample Construction Prompt

Actual prompt template

`sample_construction_audit.txt`: use `sample_construction_auditor`; audit sample construction, data flow, and variables.

Focus excerpt

Sample restrictions, attrition, variable construction, exclusions, denominators, and analysis-sample consistency.

[Back to agent overview](#)

Appendix: Abstract/Conclusion Prompt

Actual prompt template

```
abstract_conclusion_consistency_audit.txt: use  
abstract_conclusion_consistency_auditor.
```

Focus excerpt

Audit consistency between summary sections and the manuscript body: abstract, introduction, conclusion, and central body evidence.

[Back to agent overview](#)

Appendix: Limitations Prompt

Actual prompt template

```
limitations_external_validity_audit.txt: use  
limitations_external_validity_auditor.
```

Focus excerpt

Limitations, scope, external validity, policy implications, welfare claims, and generalizability.

[Back to agent overview](#)

Appendix: Model/Equation Prompt

Actual prompt template

`model_equation_audit.txt`: use `model_equation_auditor`; audit model, equation, notation, and text-equation consistency only.

Focus excerpt

Equation definitions, notation drift, sign conventions, assumptions, and whether text claims match equations.

[Back to agent overview](#)

Appendix: Replication Prompt

Actual prompt template

```
data_availability_replication_audit.txt: use  
data_availability_replication_auditor.
```

Focus excerpt

Preregistration, pre-analysis-plan documentation, planned-versus-reported analyses, replication materials, and scoped reproducibility claims.

[Back to agent overview](#)

Appendix: Institutional Context Prompt

Actual prompt template

`institutional_context_audit.txt: use institutional_context_auditor.`

Focus excerpt

Material institutional, legal, regulatory, platform, market-design, and event-context claims.

[Back to agent overview](#)

Appendix: Power/Multiplicity Prompt

Actual prompt template

```
power_multiple_testing_audit.txt: use power_multiple_testing_auditor.
```

Focus excerpt

Power, outcome families, multiplicity, exploratory-versus-confirmatory status, and strong null-effect interpretation.

[Back to agent overview](#)

Appendix: Design/Randomization Prompt

Actual prompt template

`design_randomization_audit.txt: use design_randomization_auditor.`

Focus excerpt

Experimental assignment, randomization, treatment implementation, post-assignment attrition, spillovers, contamination, and fidelity.

[Back to agent overview](#)

Appendix: Economic Magnitude Prompt

Actual prompt template

`economic_magnitude_audit.txt: use economic_magnitude_auditor.`

Focus excerpt

Practical significance, economic magnitude, baselines, units, denominators, willingness-to-pay, welfare, cost, and calibration interpretation.

[Back to agent overview](#)

Appendix: Editor Bundle Summary

```
"summary": {  
  "canonical_finding_count": 60,  
  "issue_class_counts": {  
    "parser_artifact": 8,  
    "reference_integrity": 6,  
    "copyedit_issue": 9,  
    "manuscript_issue": 31,  
    "cannot_verify": 5  
  },  
  "severity_counts": {  
    "medium": 47,  
    "low": 9,  
    "high": 4  
  }  
}
```

[Back to normalization](#)

Appendix: Normalization Scripts

```
python scripts\normalize_review_outputs.py ^  
  --paper-id paper3 ^  
  --reviews-dir work\paper3\reviews ^  
  --output work\paper3\editor\normalized_bundle.json
```

```
python scripts\build_editor_input.py ^  
  --paper-id paper3 ^  
  --bundle work\paper3\editor\normalized_bundle.json
```

Role

These scripts merge reviewer JSON, deduplicate findings, and create the deterministic editor brief.

[Back to normalization](#)

Appendix: Report Smoke Check

```
python scripts\check_final_report.py ^  
  --input outputs\paper3\report.md ^  
  --bundle work\paper3\editor\normalized_bundle.json
```

What it checks

The report exists, expected sections are present, and traceability IDs from the normalized bundle are represented.

[Back to workflow](#)

Backup: Static Versus Dynamic Selection

Dynamic mode

The selector chooses optional reviewers based on paper type and parsed evidence.

Static mode

All enabled review-stage reviewers run. This is useful for debugging or exhaustive tests.

[Back to workflow](#)

Backup: What To Add For A New Reviewer

1. Custom agent definition in `.codex/agents/`.
2. Prompt template in `prompts/templates/`.
3. Reviewer entry in `config/reviewers.json`.
4. Schema-compatible output behavior.
5. Normalization and validation tests if needed.

[Back to agent overview](#)